# Makanak Real Estate Platform

## Project Scope & Requirements

*Week 1 Deliverables*

# 1. Project Scope Document

## 1.1 Project Vision & Objectives

Makanak is a Full-Stack .NET Real Estate Platform designed to connect property owners with buyers and tenants. The platform allows users to browse properties, search and filter listings, and negotiate prices directly through a simple offer management system.

The main objective is to deliver a functional Minimum Viable Product (MVP) within four weeks, focusing on the essential features without unnecessary complexity.

## 1.2 Project Scope (MVP)

### 1. Authentication & Authorization

- User registration and login using ASP.NET Core Identity and JWT.
- Three user roles: Buyer/Tenant, Owner, and Admin.
- Role-based authorization to control access to system features.

### 2. Property Management

- Full CRUD operations for property listings.
- Upload and manage property images, including a primary image.
- Support property types such as apartments, villas, and offices.
- Specify listing type: Sale or Rent.
- Store property details, including title, description, price, address, and city.
- Manage property status: Available, Sold, Rented, or Inactive.

### 3. Property Search & Filtering

- Search properties using keywords.
- Filter properties by price range, property type, listing type, and city.
- Sort results by price or creation date.
- Support pagination.

### 4. Offers & Negotiation

- Buyers/Tenants can submit price offers on available properties.
- Owners can accept, reject, or submit counter-offers.
- Buyers/Tenants can respond to counter-offers.
- Maintain a negotiation history for each offer.
- Prevent new offers on unavailable properties and prevent conflicting acceptances.

### 5. User Dashboards

- Owner Dashboard for managing properties and reviewing received offers.
- Buyer/Tenant Dashboard for tracking submitted offers and their statuses.
- Display negotiation history and current offer status.

## 1.3 Out of Scope

- Real-time notifications using SignalR or WebSockets.

- Online payment gateways.
- Interactive maps and external GPS integrations.
- Email and SMS notification systems.
- Advanced analytics and reporting.
- Direct messaging between users outside the offer negotiation process.

# 2. Functional & Non-Functional Requirements

## 2.1 Functional Requirements

| ID | Requirement |
|---|---|
| FR-01 | The system allows users to register and log in using JWT authentication. |
| FR-02 | The system supports Buyer/Tenant, Owner, and Admin roles with appropriate access permissions. |
| FR-03 | Owners can create, view, update, and delete their own property listings. |
| FR-04 | Owners can upload property images and select a primary image. |
| FR-05 | Users can browse, search, filter, sort, and paginate property listings. |
| FR-06 | Buyers/Tenants can submit price offers on available properties they do not own. |
| FR-07 | Owners can accept, reject, or submit counter-offers for offers on their properties. |
| FR-08 | Buyers/Tenants can respond to counter-offers by accepting, rejecting, or submitting another counter-offer. |
| FR-09 | The system records negotiation history, including the user, action, price, and timestamp. |
| FR-10 | Buyers/Tenants can view their submitted offers and current statuses through their dashboard. |
| FR-11 | Owners can manage their properties and view received offers through their dashboard. |
| FR-12 | The system prevents unauthorized users from modifying properties or offers they do not own or control. |
| FR-13 | The system validates offer prices and prevents offers on unavailable properties. |
| FR-14 | The system prevents conflicting acceptances for the same property. |

## 2.2 Non-Functional Requirements

| ID | Requirement |
|---|---|
| NFR-01 — Architecture | The system uses a simplified Clean Architecture structure with separation between API, Application, Domain, and Infrastructure. |
| NFR-02 — Security | The system uses ASP.NET Core Identity for password hashing and JWT for authentication, with proper authorization and input validation. |
| NFR-03 — Data Integrity | The system maintains consistent data and prevents conflicting offer acceptances through appropriate database operations and concurrency handling. |
| NFR-04 — Usability | The user interface is responsive and follows the approved Figma designs. |
| NFR-05 — Performance | Property search and listing operations support filtering, sorting, and pagination to avoid loading all records at once. |

| | |
|---|---|
| NFR-06 — Error Handling | The system provides appropriate validation messages and consistent error responses for failed requests. |
| NFR-07 — Maintainability | The codebase follows clear naming conventions and separation of responsibilities to make development and maintenance easier. |

## 2.3 Technical Constraints

| Area | Technology / Constraint |
|---|---|
| Backend | ASP.NET Core Web API with C# |
| Database | SQL Server with Entity Framework Core |
| Authentication | ASP.NET Core Identity and JWT |
| Architecture | Simplified Clean Architecture |
| Data Access | Repository and Unit of Work, implemented without unnecessary abstraction |
| Image Storage | Basic local or configured server storage |
| Project Duration | Four weeks, with the MVP limited to the features defined in this document |